Boothflow

Technical Documentation

AI-Powered Mobile Lead Capture System

Dell Technologies Forum Singapore

Version 1.0 | July 2026

1. System Overview

Boothflow is an AI-powered mobile lead capture system designed for Dell Technologies Forum Singapore. The system enables booth representatives to scan visitor QR codes, capture lead information, and automatically analyze leads using Google Gemini 2.5 Flash AI. Managers can track leads and schedule follow-up emails, while admins oversee the entire system.

1.1 Target Users

- Booth Representatives (Rep) — Scan QR codes, capture leads, view their own leads
- Managers — View team leads, schedule follow-up emails, export reports
- Administrators (Admin) — Full system access, user management, team management

1.2 Technology Stack

Layer	Technology
Frontend	React Native, Expo Router, Expo Go, TypeScript
Backend	Node.js, Express.js, REST API
Database	Supabase (PostgreSQL)
AI	Google Gemini 2.5 Flash + Rule-Based Fallback
Email	Resend
DevOps	Docker, Docker Compose, GitHub Actions CI/CD
Hosting	Render (backend), Expo EAS (frontend)
Orchestration	Kubernetes (demo), GHCR (image registry)
Security	JWT Authentication, bcrypt, Rate Limiting, SecureStore

2. System Architecture

The system follows a three-tier architecture consisting of a mobile frontend, a RESTful backend API, and a cloud-hosted PostgreSQL database. External services include Google Gemini AI for lead analysis and Gmail for email delivery.

2.1 Architecture Overview

- Mobile App (Expo/React Native) communicates with the backend via REST API calls over HTTPS
- Backend (Node.js/Express) handles authentication, business logic, AI analysis, and email scheduling
- Database (Supabase PostgreSQL) stores all persistent data including users, leads, teams, and activity logs
- Booth Representatives also query Supabase directly for their own lead data using the anon key
- Google Gemini API is called from the backend for AI-powered lead analysis
- Resend handles automated and manual follow-up emails.
- Docker containerises the backend for consistent deployment across environments
- GitHub Actions runs automated tests and builds/pushes Docker images to GHCR on every push to main
- Render hosts the backend container with auto-deployment from GitHub
- Kubernetes (demo) demonstrates container orchestration with Horizontal Pod Autoscaling

2.2 Data Flow

Rep Flow:

Rep scans QR code → Lead data submitted to backend → Backend validates and saves to Supabase → AI analysis triggered → Gemini/Fallback returns intent score → Lead status updated → Follow-up email scheduled in lead_followups table → Cron job sends email after 3 hours

Manager Flow:

Manager logs in → JWT token issued → Manager views team leads filtered by team_id → Manager schedules manual follow-up → Backend saves to lead_followups → Cron job processes pending emails

Admin Flow:

Admin logs in → Full access to all routes → Admin manages users, teams, leads → Admin can override AI notes and lead status → Admin views system-wide activity logs

3. Database Schema

The database is hosted on Supabase (PostgreSQL). The schema consists of six core tables.

Supabase (PostgreSQL) Database Schema

1. USERS

Stores all system users (reps, managers, admins)

Columns:

- user_id (serial, PK)
- full_name (varchar)
- email (varchar, unique)
- password_hash (text)
- role (text) — rep / manager / admin
- team_id (int, FK → teams)
- is_active (bool)
- created_at (timestamptz)
- last_login (timestamptz)
- permissions (jsonb)

2. TEAMS

Stores team information for lead assignment

Columns:

- team_id (serial, PK)
- team_name (varchar)
- territory (text)
- description (text)
- created_at (timestamptz)

3. LEADS

Stores all captured leads from booth scanning

Columns:

- lead_id (serial, PK)

• name (varchar)
• company (varchar)
• title (varchar)
• email (varchar)
• phone_number (varchar)
• customer_interest (varchar)
• assigned_team_id (int, FK → teams)
• ai_notes (text)
• status (text) — NEW / CONTACTED / QUALIFIED / CLOSED
• confidence_score (float8)
• follow_up_required (bool)
• scanned_by (varchar)
• scanned_by_name (varchar)
• created_at (timestamptz)

4. LEAD_FOLLOWUPS

Tracks scheduled and sent follow-up emails

Columns:
• followup_id (serial, PK)
• lead_id (int, FK → leads, unique)
• followup_action (varchar)
• followup_status (text) — pending / done / cancelled
• due_date (date)
• notes (text)
• created_at (timestamptz)
• scheduled_at (timestamptz)
• sent_at (timestamptz)
• email_subject (text)
• updated_at (timestamptz)

5. LEAD_ACTIVITY_LOGS

Logs all lead-related activities (email sent, followup scheduled)

Columns:
• activity_id (serial, PK)

• lead_id (int, FK → leads)
• activity_type (varchar) — EMAIL_SENT / FOLLOWUP_SCHEDULED / FOLLOWUP_SENT
• activity_description (text)
• created_at (timestamptz)

6. INTEREST_CATEGORIES

Predefined interest categories (AI PCs, Storage, etc.)

Columns:
• category_id (serial, PK)
• category_name (varchar)
• description (text)
• created_at (timestamptz)

7. LEAD_INTEREST_CATEGORIES

Junction table linking leads to their interest categories

Columns:
• lead_interest_id (serial, PK)
• lead_id (int, FK → leads)
• category_id (int, FK → interest_categories)

8. SYSTEM_ACTIVITY_LOG

Tracks general system-level actions (not lead-specific)

Columns:
• activity_id (serial, PK)
• user_id (int, FK → users)
• action (text)
• entity_type (varchar)
• entity_id (int)
• description (text)
• created_at (timestamptz)

4. API Documentation

The backend exposes a RESTful API. All protected routes require a Bearer JWT token in the Authorization header. The base URL is the Render deployment URL.

4.1 Authentication Routes

Method	Endpoint	Authentication	Description
POST	/auth/login	None	Authenticate user and return a JWT token.
GET	/auth/me	Required	Retrieve the currently authenticated user's information.

4.2 Leads Routes

Method	Endpoint	Access	Description
GET	/leads	Admin,	Retrieve all leads.
POST	/leads	Admin, Booth Representative	Create a new lead.
GET	/leads/:id	Admin, Manager	Retrieve a lead by its ID.
PUT	/leads/:id	Admin, Manager, Representative	Update an existing lead.
DELETE	/leads/:id	Admin, Manager	Delete a lead and cancel any pending follow-up.

4.3 Manager Routes

Method	Endpoint	Access	Description
GET	/manager/me	Admin, Manager	Retrieve the manager's profile information.
GET	/manager/dashboard	Admin, Manager	Retrieve dashboard statistics for the manager's team.
GET	/manager/leads	Manager	Retrieve all leads assigned to the manager's team.
GET	/manager/emails	Admin, Manager	Retrieve email statistics and overdue follow-ups.
GET	/manager/activity	Authenticated users	Retrieve follow-up activity for the manager's team.
GET	/manager/export/leads	Admin, Manager	Export the manager's team leads in JSON format.
GET	/export/leads/excel	Admin, Manager	Export the manager's team leads as an Excel file.

Method	Endpoint	Access	Description
POST	/manager/followup/cancel	Manager	Cancel a scheduled follow-up.
PUT	/manager/followup/:leadId	Authenticates Users	Update the follow-up status and notes for a lead.
POST	/send-followup/:id	Manager	Send a manual follow-up email to a lead immediately.

4.4 Admin Routes

Method	Endpoint	Access	Description
GET	/admin/dashboard	Admin	Retrieve system-wide dashboard statistics.
GET	/admin/users	Admin	Retrieve all user accounts.
POST	/admin/users	Admin	Create a new user account.
POST	/admin/users/bulk	Admin	Import multiple users in a single request.
PUT	/admin/users/:id	Admin	Update an existing user's details.
DELETE	/admin/users/:id	Admin	Delete a user account.
PUT	/admin/users/:id/permissions	Admin	Update a user's permissions.
GET	/admin/leads	Admin	Retrieve all leads in the system.
PUT	/admin/leads/:id/assign	Admin	Assign or reassign a lead to a team.
PUT	/admin/leads/:id/notes	Admin	Update AI-generated notes and lead status.
GET	/admin/teams	Admin	Retrieve all teams.
POST	/admin/teams	Admin	Create a new team.
PUT	/admin/teams/:id	Admin	Update an existing team.
DELETE	/admin/teams/:id	Admin	Delete a team.
GET	/admin/activitylogs	Admin	Retrieve system activity logs.

4.5 AI & Utility Routes

Method	Endpoint	Access	Description
POST	/analyze-lead/:id	Authenticated Users	Analyse a lead using the Gemini AI model and generate follow-up

Method	Endpoint	Access	Description
			recommendations.
POST	/send-email	Authenticated Users	Schedule a follow-up email for a lead.
GET	/interest_categories	All Users	Retrieve all available interest categories.
POST	/lead_interest_categories	All Users	Associate an interest category with a lead.
GET	/lead_interest_categories/:lead_id	All Users	Retrieve the interest categories assigned to a lead.
DELETE	/lead_interest_categories/:lead_id/:category_id	All Users	Remove an interest category from a lead.
GET	/teams	All Users	Retrieve all teams.
GET	/teams/:id	All Users	Retrieve details of a specific team.
POST	/teams	All Users	Create a new team.
PUT	/teams/:id	All Users	Update an existing team.
DELETE	/teams/:id	All Users	Delete a team.
GET	/	Public	Check the API status and retrieve the list of available endpoints.

5. Setup & Installation

5.1 Prerequisites

- Node.js v20+
- npm v9+
- Expo CLI (npm install -g expo-cli)
- Docker Desktop (for local containerisation)
- Git

5.2 Environment Variables

Create a .env file in the backend/ directory with the following variables:

- DATABASE_URL — Supabase PostgreSQL connection string
- JWT_SECRET — Secret key for signing JWT tokens
- GEMINI_API_KEY — Google Gemini API key
- EMAIL_USER — Gmail address for sending follow-up emails
- EMAIL_PASS — Gmail App Password (16-character, not regular password)
- PORT — Server port (default: 3000)

Create a .env file in the root directory with:

- EXPO_PUBLIC_BACKEND_URL — Render backend URL
- EXPO_PUBLIC_API_URL — Supabase REST API URL
- EXPO_PUBLIC_SUPABASE_ANON_KEY — Supabase anonymous key

5.3 Running the Backend

Standard (Node.js):

```
cd backend && npm install && node app.js
```

Docker Compose:

```
cd backend && docker compose up
```

5.4 Running the Frontend

Prerequisites

- Node.js 20 or later
- npm or yarn
- Expo Go app installed on Android device
- Expo CLI (npm install -g expo-cli)

Installation

```
bash
# Install dependencies
npm install --legacy-peer-deps

# Fix any SDK 55 package version conflicts
npx expo install --fix -- --legacy-peer-deps

# Start the development server
npx expo start
```

Running the App

- Scan the QR code with **Expo Go** on your Android device
- Or press a in the terminal to open in Android emulator (requires Android Studio)

Building the APK

```
bash
```

```
# Login to EAS
eas login

# Build APK (preview profile)
eas build --profile preview --platform android
```

6. DevOps & Deployment

6.1 Docker

The backend is containerised using Docker. The Dockerfile uses a node:20-alpine base image, installs production dependencies, and runs app.js. Docker Compose is used for local development and CI testing, spinning up the backend container with all required environment variables.

6.2 CI/CD Pipeline

Two GitHub Actions workflows are configured:

Backend CI/CD (ci.yml):

- Install backend dependencies
- Install Postman CLI
- Start backend via Docker Compose
- Run health check against localhost:3000
- Run 46 Postman test cases (143 assertions)
- Build and push Docker image to GHCR on merge to main

Frontend CI (frontend.yml):

- Install dependencies with --legacy-peer-deps
- TypeScript type checking
- ESLint linting
- Jest test runner
- Expo config validation

6.3 Render Deployment

Render Deployment

Prerequisites

- Render account at render.com
- GitHub repository connected to Render
- All environment variables ready

Steps

1. Go to **render.com** → New → **Web Service**

2. Connect your GitHub repository
3. Configure the service:
    a. **Name:** boothflow-backend
    b. **Root Directory:** backend
    c. **Runtime:** Node
    d. **Build Command:** npm install
    e. **Start Command:** node app.js
4. Add Environment Variables under **Settings → Environment:**
   DATABASE_URL=your-supabase-connection-string
   JWT_SECRET=your-jwt-secret
   GEMINI_API_KEY=your-gemini-api-key
   RESEND_API_KEY=your-resend-api-key
   RESEND_FROM_EMAIL= your domain or the resend test domain
   PORT=3000
5. Click **Deploy**

Health Check

- Set Health Check Path to /
- Render pings this every few minutes to keep the service alive

Note: Render free tier services spin down after inactivity. The first request after inactivity may take 30–60 seconds to respond as the service restarts.

UptimeRobot pings the server every 5 minutes to prevent Render's free tier from sleeping, ensuring 24/7 availability.

6.4 Kubernetes (Local Demo)

Kubernetes is configured using Docker Desktop on a team member's laptop. The deployment.yaml pulls the backend Docker image from GHCR and deploys it as a pod. Horizontal Pod Autoscaling (HPA) is configured to scale pods based on CPU usage. Freelens is used to visualise the cluster during the demo. In a production environment, this would be deployed on a cloud Kubernetes provider such as GKE or AKS.

Here is the Boothflow Kubernetes Deployment guide:

BoothFlow Kubernetes Deployment Guide (Final).pdf

7. Security

7.1 Authentication — JWT

JSON Web Tokens (JWT) are used for stateless authentication. On login, the server signs a token containing the user's ID, role, and team_id with a secret key. The token expires after 8 hours. All protected API routes validate the token using the authenticateToken middleware.

7.2 Role-Based Access Control (RBAC)

Role	Access
rep	Own leads only (filtered by scanned_by via Supabase direct query)
manager	/manager/* routes — team leads filtered by team_id from JWT
admin	All routes — full system access, no data filtering

7.3 Password Security

Passwords are hashed using bcrypt with a salt factor of 10 before storage. Plain text passwords are never stored in the database. Password comparison is done using bcrypt.compare().

7.4 Rate Limiting

- General routes: 100 requests per 15 minutes per IP
- Authentication routes (/auth/*): 20 requests per 15 minutes per IP — prevents brute force attacks

7.5 Token Storage

On the frontend, JWT tokens are stored using Expo SecureStore, which uses the device's secure keychain (iOS Keychain / Android Keystore). Tokens are never stored in AsyncStorage or memory only.

7.6 PDPA Compliance

- Booth reps can only edit customer_intent and additional_notes — personal details (name, email, phone) are read-only after capture
- Admin can edit operational data (status, AI notes, team assignment) but not personal contact details
- Email and phone number are masked in the rep view using first-two/last-two character masking
- All lead data is stored securely in Supabase with SSL-encrypted connections

7.7 Additional Backend Security Features

Environment Variables

- Stores sensitive information such as API keys, database URLs, and JWT secrets outside the source code.
- Prevents confidential credentials from being exposed if the code is shared.

Input Validation

- Validates user input before processing.
- Checks required fields, email format, and phone number format.
- Prevents invalid or incomplete data from being stored in the database.

Secure Database Queries (Parameterized Queries)

- Uses parameterized SQL queries instead of directly inserting user input into SQL statements.
- Protects the application against SQL Injection attacks.

SSL Database Connection

- Encrypts communication between the backend server and the database.
- Helps prevent data from being intercepted during transmission.

Error Handling

- Uses try...catch blocks and a global error handler to manage unexpected errors.
- Prevents the server from crashing and returns controlled error messages to users.

8. AI Integration

8.1 Google Gemini 2.5 Flash

When a lead is created, the system calls the POST /analyze-lead/:id endpoint. The backend retrieves the lead information from the database and constructs a prompt containing the lead's name, company, job title, customer intent, selected interest categories, any additional interests provided by the user, and representative notes (if available). The prompt is sent to Gemini 2.5 Flash to generate an AI assessment of the lead. Gemini 2.5 Flash returns a JSON response with:

- `intent — Low, Medium, or High`
- `confidence — float between 0 and 1`
- `follow_up_required — boolean`
- `notes — personalised 1-2 sentence follow-up suggestion`

8.2 Lead Status Assignment

Intent	Confidence	Follow-up Required	→ Status
High	≥ 0.8	Yes	URGENT
High	< 0.8	Yes/No	FOLLOW-UP
Medium	Any	Yes	FOLLOW-UP
Low	Any	Any	NEW

8.3 Rule-Based Fallback

If the Gemini API request fails due to network errors, quota limitations, or an invalid AI response, the backend automatically switches to a rule-based analysis. The system analyses the customer's selected intent using predefined keywords such as "Ready for Follow-up", "Pricing", and "Demo" to determine the lead's intent level, confidence score, follow-up requirement, and lead status. This ensures that lead qualification continues to function even when the AI service is unavailable.

8.4 Interest → Team Mapping

Primary Interest	Assigned Team
AI PCs	Team 1
Multi-cloud	Team 2
Storage	Team 3
Service	Team 4
Others / No match	Team 5

8.5 Automated Follow-up Scheduling

After the AI analysis is completed, the system automatically creates a pending follow-up email for the lead if one does not already exist. The email is scheduled to be sent three hours after analysis and is stored in the lead_followups table. A cron job executes every five minutes to identify pending follow-ups that are due and sends the email automatically. Once the email has been successfully delivered, the follow-up status is updated to **Done**, and the lead status is changed to **Closed**.

9. Testing

9.1 API Testing — Postman

The backend is tested using a Postman collection with **70 test cases covering 246 assertions**. Tests are organised into **10 folders** across **74 requests**, all executed with **0 failures.**

- **00 Auth** — Login (rep, manager, admin, invalid credentials), Get Me
- **01 Leads** — Create, Get All, Get by ID, Create with missing fields, invalid email, duplicate email
- **02 Interest Categories** — Add, Get, Remove interests for a lead
- **03 AI Analysis** — Analyse lead, lead not found
- **04 Teams** — CRUD operations for teams
- **05 Manager** — Dashboard, leads, emails, export, follow-up email, activity log, access control
- **06 Admin** — Users CRUD, bulk import, permissions, teams, activity logs, access control
- **07 Cleanup** — Update lead, delete test admin lead, delete bulk created users, delete Jane Doe from leads, delete test admin user, verify lead not found after deletion
- **08 Email Follow-up** — Schedule follow-up emails, duplicate check, manual send, status updates after sending
- **09 Manager Follow-up Management** — Update follow-up status, cancel follow-up, access control validation

Tests are run automatically in CI/CD via **Newman CLI** against the live Render backend. Authentication tokens are pre-fetched via PowerShell before the test run and passed as --env-var flags, so no environment file is needed in the CI environment.

9.2 Running Tests on Terminal

```
.\scripts\run-tests.ps1
```

9.3 Frontend Validation Testing — Jest

Frontend validation tests are written using Jest and @testing-library/react-native. Tests cover form validation, input handling, and component rendering for key screens.

10. Known Limitations

- Render free tier limits backend to 750 compute hours per month. UptimeRobot pings prevent sleeping but the server may briefly sleep at month end.

- The application uses a cron job that runs every five minutes to process scheduled follow-up emails. While suitable for this prototype, a production deployment should use a dedicated background job queue (e.g., BullMQ with Redis) for improved reliability, scalability, and fault tolerance.

- JWT tokens expire after 8 hours. Users must re-login after expiry.
- The application depends on the Gemini 2.5 Flash API for AI lead analysis. If the API is unavailable because of rate limits, network failures, or invalid responses, the system automatically falls back to a rule-based analysis to ensure uninterrupted functionality.
- Kubernetes demo is not a production deployment. Cloud K8s would require additional configuration (Follow the K8s deployment guide).
- The React Native version conflict between react-native 0.83.6 and newer peer dependencies is resolved using npm overrides. This may require attention when upgrading dependencies.
- Scheduled follow-up emails rely on the backend server remaining online. If the server becomes unavailable during the scheduled delivery time, emails will be delayed until the application is running again.
- AI-generated lead analysis is dependent on the quality and completeness of the information provided by the sales representative. Incomplete or inaccurate lead details may reduce the accuracy of the generated recommendations.
- Expo Go is set to SDK 55 for this project and only android users can have access to SDK 55. There are also limitations as there are multiple package conflicts.